

FREE · FROM THE BOOK

The Cheatsheets

Every chapter of *Math for ML Engineers* on a page or two:
the key results, the decision guide, and the numbers to know cold.

Math for ML Engineers

FROM BACKPROP TO RLHF · THE DERIVATIONS BEHIND MODERN AI

Mikael Yemane · ISBN 979-8-9974520-0-1

mikaelyemane.github.io/ml-math-code

© 2026 Mikael Yemane. All rights reserved. This bundle may be shared freely in unmodified form.

Contents

The Cheat-Sheet	2
Numbers You Should Know Cold	4
1 Mathematical Foundations	9
2 Probability and Statistics	11
3 Information Theory for Machine Learning	12
4 Linear Models and Their Geometry	13
5 Gradient Descent and the Optimization Landscape	14
6 Kernel Methods and Feature Spaces	16
7 Shallow Neural Networks	17
8 Deep Networks: Backpropagation and Modern Architectures	19
9 Convolutional Neural Networks	21
10 Recurrent Networks: Sequence Modeling Through Time	23
11 Attention and Transformers	25
12 Generative Models	27
13 Reinforcement Learning: Optimization Under Uncertainty	28
14 Frontiers: Where Mathematics Meets Research	29
15 Production Inference Economics	31
16 Mechanistic Interpretability	33
17 Abstraction and Interface Design in ML Systems	35
18 Testing, Versioning, and Reproducibility for ML Engineers	38
19 Technical Debt and Observability in Production ML	40

Each chapter page collects that chapter’s summary box and decision guide exactly as they appear in the book; section and equation numbers point to the full text. Chapter 5 is free in full at mikaelyemane.github.io/ml-math-code.

The Cheat-Sheet

The eleven results from *What every ML engineer should know cold* (see *How to Read This Book*), restated as one-line identities you can reproduce on a whiteboard. Take this sheet into an interview; do not let yourself out of it without being able to derive each. (Appendix F's *The Ten Non-Negotiables* covers much of the same ground, framed as interview questions with a chapter pointer rather than identities to recall.)

1. SVD and Eckart–Young. For $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, the best rank- k approximation in spectral or Frobenius norm is $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$.

2. MLE = NLL = Cross-Entropy. Maximizing $\sum_i \log p_\theta(\mathbf{y}_i | \mathbf{x}_i)$ is minimizing the negative log-likelihood, which for a categorical model equals $\sum_i \text{CE}(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \sum_i H(\mathbf{y}_i) + \text{KL}(\mathbf{y}_i \| \hat{\mathbf{y}}_i)$.

3. Softmax cross-entropy gradient. For logits $\mathbf{z}$, target one-hot $\mathbf{y}$, and prediction $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$: $\nabla_{\mathbf{z}} \text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = \hat{\mathbf{y}} - \mathbf{y}$.

4. Backpropagation = reverse-mode AD. For $\mathcal{L} = f_L \circ \dots \circ f_1(\mathbf{x})$, the gradient

$$\partial \mathcal{L} / \partial \boldsymbol{\theta}_l = (\partial f_l / \partial \boldsymbol{\theta}_l)^\top \mathbf{J}_{l+1}^\top \dots \mathbf{J}_L^\top$$

is accumulated right-to-left in $O(\text{forward cost})$.

5. Adam.

$$\hat{\mathbf{m}}^{(t)} = \frac{\mathbf{m}^{(t)}}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}^{(t)} = \frac{\mathbf{v}^{(t)}}{1 - \beta_2^t}, \quad \boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\eta \hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \epsilon}}.$$

The bias correction exactly removes the zero-initialization bias when gradient statistics are stationary; per-coordinate adaptivity is the leading explanation for why it dominates Transformer training.

6. LSTM cell-state highway. $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ has additive update along $\mathbf{c}$, so along the direct cell path $\partial \mathbf{c}_T / \partial \mathbf{c}_0 \approx \prod_t \mathbf{f}_t$ which can stay near $\mathbf{1}$ throughout the sequence.

7. Scaled dot-product attention. $\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k})\mathbf{V}$. The $\sqrt{d_k}$ keeps the pre-softmax logits at unit variance so the softmax does not saturate; the softmax weights then average the rows of $\mathbf{V}$: a soft lookup keyed by query–key similarity.

8. ELBO. $\log p_\theta(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \text{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$ with equality iff $q_\phi = p_\theta(\cdot|\mathbf{x})$. For Gaussian q and standard-Gaussian prior, the KL term is closed form (Chapter 3).

9. DDPM.

$$\text{Forward: } \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}, \quad \text{Loss: } \mathbb{E}_{\boldsymbol{\varepsilon}, t} \|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2.$$

Reverse: subtract the predicted noise scaled by the schedule.

10. DPO from RLHF. The RLHF KL-constrained optimal policy satisfies $\log \pi^* / \pi_{\text{ref}} = (1/\beta)r(x, y) + \text{const}$; substituting into the Bradley–Terry preference likelihood yields $\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \log \sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})$.

11. Chinchilla allocation. For fixed compute $C = 6ND$, optimal $N^* \propto C^{\beta/(\alpha+\beta)}$ and $D^* \propto C^{\alpha/(\alpha+\beta)}$; the symmetric simplification $\alpha = \beta$ fixes only the exponents ($N^*, D^* \propto C^{0.5}$) — the practitioner shorthand $D/N \approx 20$ is set by the fitted constants A and B , not by the exponent symmetry.

If you can derive each of these from scratch, you have absorbed the core of this book.

Numbers You Should Know Cold

These are the empirical constants, production defaults, and back-of-envelope thresholds that experienced ML engineers know without looking them up. Each number is annotated with the section that derives or motivates it. A question mark (?) means the number is context-dependent; the range given is the typical production default.

Hardware

Quantity	Value	Reference
A100-80G bf16 throughput	312 TFLOP/s	Chapter 15
A100-80G HBM bandwidth	2.0 TB/s	Chapter 15
A100 ridge point	≈ 156 FLOP/byte	Chapter 15
H100-SXM5 bf16 throughput	989 TFLOP/s	Chapter 15
H100-SXM5 HBM bandwidth	3.35 TB/s	Chapter 15
SRAM (L1 + shared, A100)	192 KB/SM	≤ 164 KB usable as shared; Section 11.9
bf16 max finite value	$\approx 3.4 \times 10^{38}$	Section 8.9
fp16 max finite value	65,504 ($\approx 6.55 \times 10^4$)	Section 8.9
fp16 overflow threshold	$\approx e^{11.09}$	Section 8.9

Optimizers (Chapter 5)

Quantity	Default	Notes
Adam β_1	0.9	Standard
Adam β_2	0.999	Standard; 0.95 for Transformers
Adam ϵ	10^{-8}	10^{-6} for fp16 (underflow); often also bf16
Adam weight decay (AdamW)	0.1	Decoupled, not L_2 penalty
Gradient clip norm	1.0	Transformer standard
Warmup steps	1K–10K	$\approx 2/(1 - \beta_2)$ at $\beta_2=0.999$; Section 5.5
Cosine schedule min LR	$0.1 \times$ peak LR	
μ P hidden-weight LR multiplier	$\propto 1/d$	Section 5.6

Transformer Architecture (Chapter 11)

Quantity	Value	Notes
Params per Transformer block	$\approx 12d^2$	Excl. embeddings
FFN to attention param ratio	$2\times$	$8d^2$ vs. $4d^2$
Llama-3-8B: d, H, G, d_k, L	4096, 32, 8, 128, 32	
Llama-3-70B: d, H, G, d_k, L	8192, 64, 8, 128, 80	
GPT-2 (1.5B): d, H, d_k, L	1600, 25, 64, 48	MHA ($G = H$)
GQA memory saving	$H/G\times$	vs. full MHA
FlashAttention wall-clock gain	$2\text{--}4\times$	Same FLOPs
FlashAttention memory	$O(nd)$ vs. $O(n^2)$	Per-layer activations
KV cache: Llama-3-70B, 128K ctx	≈ 43 GB/request	bf16, GQA $G = 8$
KV cache per token, Llama-3-70B	≈ 328 KB	bf16; = 320 KiB; know this cold
Sliding-window: Mistral-7B w	4,096	Fixed regardless of n
RoPE base: Llama-3	500,000	$50\times$ original 10,000

Training Scale (Chapter 14)

Quantity	Value	Notes
Chinchilla: optimal tokens/param	≈ 20	At fixed compute budget
Chinchilla exponents (α, β)	$(0.34, 0.28)$	Hoffmann et al. 2022
Compute, 1B params, 20B tokens	$\approx 1.2\times 10^{20}$ FLOP	$C \approx 6ND$
FLOPs per token, dense LLM (inference)	$\approx 2P$	P = parameter count
FLOPs per token, dense LLM (training)	$\approx 6P$	Fwd + bwd; matches $C \approx 6ND$
Double-descent peak location	$N \approx n$ (params $\approx$ data)	Chapter 14

Quantization (Chapters 8 and 14)

Format	Bits	Notes
fp32	32	1 sign, 8 exp, 23 mantissa
bf16	16	1 sign, 8 exp, 7 mantissa
fp16	16	1 sign, 5 exp, 10 mantissa; overflows
fp8 E4M3	8	Activations; range ± 448
fp8 E5M2	8	Gradients; wider range
INT8 weight-only	8	$2\times$ memory reduction vs. bf16 ($\frac{1}{2}$ the footprint)
INT4 (GPTQ/AWQ)	4	$4\times$ memory reduction vs. bf16 ($\frac{1}{4}$ the footprint); < 1 PPL loss
Llama-3-70B bf16	140 GB	2 A100-80G minimum
Llama-3-70B INT4	≈ 35 GB	Fits on 1 A100-80G

Fine-tuning (Chapters 11 and 14)

Quantity	Default	Notes
LoRA rank r (starting point)	16	On $\mathbf{W}^Q, \mathbf{W}^V$
LoRA α / scaling	r or $2r$	$\Delta \mathbf{W}$ scaled by α/r
QLoRA quantization	4-bit NF4 base	+ bf16 adapters
Llama-3-70B LoRA adapter size	≈ 400 MB	$r = 16$, all proj layers, bf16
Full fine-tune 7B memory	≈ 112 GB	Weights + optimizer state
LoRA fine-tune 7B memory	≈ 16 GB	bf16 base + adapters; 1 A100-40G

Alignment (Chapter 13)

Quantity	Default	Notes
DPO β	0.1	KL penalty weight
GRPO group size G	16	Rollouts per prompt
PPO clip range ϵ	0.2	Policy update clipping
SFT then alignment tokens	10B+ SFT; 1B+ pref	Rough scale

Generative Models (Chapter 12)

Quantity	Default	Notes
DDPM diffusion steps (training)	1,000	
DDIM steps (inference)	50	Near-DDPM quality
Flow matching steps (inference)	8–20	Straighter path
CFG guidance scale w (SD)	7.5	Stable Diffusion default
CFG condition dropout rate	0.1–0.2	During training
CLIP temperature τ	0.07	CLIP init (learned)
CLIP optimal batch size	$\geq 32K$	Quality grows with $\log(\text{batch size})$
VAE active latents	$> \frac{1}{2}$ of dims	per-dim $KL_j \not\approx 0$, §12.2

Language Model Quality (Chapter 3)

Cross-entropy (nats/token)	PPL	Quality tier
≈ 4.95	≈ 141	KN-smoothed 5-gram (PTB)
≈ 2.86	≈ 17.5	GPT-2 1.5B (WikiText-103, zero-shot)
≈ 1.4	≈ 4	Frontier LLM (2025+)
<i>LLM inference sampling defaults</i>		
Temperature	0.6–1.0	Chat models
Top- p (nucleus)	0.85–0.95	Chat models
Temperature 0 (greedy)	—	Deterministic tasks

Mechanistic Interpretability (Chapter 16)

Quantity	Typical value	Notes
SAE dictionary size m vs. d	$8\times\text{--}64\times$	Overcomplete factor
SAE sparsity penalty λ	problem-specific	Tune via recon/sparsity Pareto
Dead latents, unmitigated	up to 90%	See the Key Idea in §16.2.1
Near-orthogonal packing	$k \gtrsim e^{cd\epsilon^2}$	Directions in $\mathbb{R}^d$, Theorem 16.1

Production Testing & Monitoring (Chapters 18 and 19)

Quantity	Typical value	Notes
ML Test Score, per-test rating	0 / 0.5 / 1	Chapter 18, minimum across 4 categories
PSI: no material shift	< 0.1 ?	Chapter 19, rule of thumb, not universal
PSI: moderate shift	0.1–0.25 ?	Chapter 19, worth investigating
PSI: substantial shift	> 0.25 ?	Chapter 19, treat as actionable

Values are approximate, as of 2026; check the model card and hardware spec for your deployment. The PSI bands are rules of thumb; Chapter 19 shows how to set each threshold from the feature’s own history (an empirical null).

CHAPTER 1

Mathematical Foundations

KEY IDEA · Chapter Summary**Key results:**

1. Every real symmetric matrix has an orthonormal eigenbasis: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (Spectral Theorem).
2. Every matrix has an SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$; the singular values measure how much each direction is stretched, and the truncated SVD gives the best low-rank approximation (Eckart–Young).
3. The gradient of $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ is $(\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$; for symmetric $\mathbf{A}$ this simplifies to $2\mathbf{A} \mathbf{x}$. The alternative $\frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}$ has gradient $\mathbf{A} \mathbf{x}$ for symmetric $\mathbf{A}$ (the $\frac{1}{2}$ cancels the 2).
4. A twice-differentiable function is convex if and only if its Hessian is PSD everywhere; strong convexity ($\nabla^2 f \succeq \mu \mathbf{I}$, $\mu > 0$) gives a unique minimizer and fast gradient-descent convergence.

Running example: The SVD of the 500×2 exam-score design matrix revealed that study hours (x_1) dominates the first singular direction $\mathbf{v}_1 \approx (0.95, 0.30)$, capturing $\approx 95\%$ of the total squared singular-value mass (consistent with x_1 's larger range $[0, 10]$ vs x_2 's $[0, 4]$); GPA has the larger per-unit coefficient (10 vs 5), but study hours contributes more of the score's variance ($\text{Var}[5x_1] \approx 208$ vs $\text{Var}[10x_2] \approx 133$) because of its larger range.

DECISION GUIDE · Which Matrix Decomposition?

<i>You need...</i>	<i>Use...</i>
PCA, low-rank approximation, LoRA	SVD: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$; keep top k singular values. Eckart–Young guarantees best rank- k fit. (Section 1.4)
Solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ where $\mathbf{A}$ is symmetric positive definite	Cholesky: $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$, two triangular solves. $\sim n^3/3$ vs. $\sim 2n^3/3$ flops for LU — roughly a 1.3–1.5 $\times$ speedup in practice (the 2 $\times$ theoretical speedup is FLOPs-only; memory-bandwidth limits and comparable BLAS optimization for LU reduce the practical gain).
Spectral clustering, PageRank, Markov chains	Eigendecomposition: $\mathbf{A}\mathbf{q}_i = \lambda_i\mathbf{q}_i$. Valid for any diagonalizable $\mathbf{A}$ (symmetric $\mathbf{A}$ is always diagonalizable; PageRank and Markov matrices are non-symmetric stochastic — diagonalizable in the typical case, but not always; the general tool is Perron–Frobenius theory). (Section 1.3)
Gradient of $\mathbf{w}^\top \mathbf{A}\mathbf{w}$ or $\ \mathbf{A}\mathbf{x} - \mathbf{b}\ ^2$	Matrix calculus identities: $\nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{A}\mathbf{w}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{w}$. (Section 1.5)
Numerical stability concern in fp16	log-sum-exp trick: subtract max before exponentiating. Prevents overflow; fp16 max $\approx 65,504$, reached at inputs $x \gtrsim 11.1$. (Section 1.7)

CHAPTER 2

Probability and Statistics

KEY IDEA · Chapter Summary

Key results:

1. MLE maximizes the likelihood; MAP adds a log-prior penalty. MAP is the posterior mode; MLE is MAP under a flat prior.
2. The LLN guarantees that empirical averages converge to expectations; the CLT characterizes the $O(1/\sqrt{n})$ fluctuations.
3. The Cramér–Rao bound establishes that no unbiased estimator can have variance below $1/(n I(\theta))$ given n i.i.d. observations; the MLE achieves this bound asymptotically.

Running example: fitting $y \sim \mathcal{N}(\mu, \sigma^2)$ to the exam scores by MLE gave the plug-in estimates $\hat{\mu} = \bar{y}$ and $\hat{\sigma}^2$; a Gaussian prior on the mean would shrink $\hat{\mu}$ toward the prior mean, the same MAP-versus-MLE trade-off that reappears with real numbers once the exam score model gains covariates x_1, x_2 with weights (w_1, w_2) in Chapter 4, previewing ridge regression there.

DECISION GUIDE · Which Estimation Strategy?

<i>You need...</i>	<i>Use...</i>
Point estimate, abundant data, no prior	MLE: $\hat{\theta} = \arg \max_{\theta} \log p(\mathcal{D} \theta)$. Equivalent to minimizing KL divergence from data. (Section 2.4)
Regularization, informative prior	MAP: adds $\log p(\theta)$ to the objective. Gaussian prior $\Rightarrow \ell_2$ penalty; Laplace $\Rightarrow \ell_1$. (Section 2.5)
Full uncertainty quantification	Bayesian posterior $p(\theta \mathcal{D})$. Conjugate priors (Beta–Bernoulli, Dirichlet–Categorical) are closed-form. (Section 2.5)
Sample size for an eval at 1% precision, 95% CI	$n \geq 18,500$ for binary metric. Hoeffding: $n \geq \ln(2/\delta)/(2\epsilon^2)$ (for binary/bounded- $[0, 1]$ outcomes; multiply by $(b - a)^2$ for general $[a, b]$ random variables). (Section 2.6)
Differentiable discrete sampling (discrete VAEs, categorical policies)	Gumbel-softmax: $\hat{y}_k = \text{softmax}((\mathbf{z} + \mathbf{g})/\tau)_k$, where z_k are logits and $g_k \sim \text{Gumbel}(0, 1)$. Straight-through estimator for backprop. (Section 2.3.3)

CHAPTER 3

Information Theory for Machine Learning

KEY IDEA · Chapter Summary

Key results: (1) Entropy $H(p)$ measures average uncertainty; KL divergence $\text{KL}(p\|q) \geq 0$ measures how much q misrepresents p . (2) Cross-entropy decomposes as $H(p, q) = H(p) + \text{KL}(p\|q)$; minimizing cross-entropy loss is equivalent to minimizing KL divergence to the data distribution. (3) Pinsker’s inequality connects KL divergence to total variation: $\text{TV}(p, q) \leq \sqrt{\frac{1}{2} \text{KL}(p\|q)}$. (4) The data processing inequality: $\text{MI}(X; Z) \leq \text{MI}(X; Y)$ for any $X \rightarrow Y \rightarrow Z$ Markov chain.

Running example: The exam score pass/fail label has $H(Y) \approx 0.78$ bits. Study hours carries more mutual information with the outcome than GPA ($\text{MI}(X_1; Y) \approx 0.27$ vs $\text{MI}(X_2; Y) \approx 0.17$ bits), consistent with the variance comparison in Chapter 1 ($\text{Var}[5X_1] \approx 208$ vs $\text{Var}[10X_2] \approx 133$), even though GPA has the larger per-unit true coefficient.

DECISION GUIDE · Which Loss Function for Your Model?

<i>Your output is...</i>	<i>Loss is...</i>
Discrete / categorical (classification, tokens)	Cross-entropy $H(p, q) = -\sum p_i \log q_i$. Minimizing CE $\equiv$ minimizing $\text{KL}(p\ q)$. (Section 3.6)
Continuous, Gaussian noise assumed	$\text{MSE} = \frac{1}{n} \ \mathbf{y} - \hat{\mathbf{y}}\ ^2$. Equivalent to MLE under $\mathcal{N}(f(\mathbf{x}), \sigma^2)$. (Chapter 2)
Latent variable model (VAE, diffusion)	$\text{ELBO} = \mathbb{E}[\log p(\mathbf{x} \mathbf{z})] - \text{KL}(q(\mathbf{z} \mathbf{x}) \ p(\mathbf{z}))$. Lower bound on $\log p(\mathbf{x})$; gap $= \text{KL}(q(\mathbf{z} \mathbf{x}) \ p(\mathbf{z} \mathbf{x}))$. (Chapter 12)
Language model quality tier	Perplexity $= e^{H(p, q)}$. Frontier LLMs: ≈ 4 PPL (1.4 nats/token) on in-distribution benchmarks (e.g., Pile subsets; WikiText-103 gives ≈ 6 –20 PPL). GPT-2 (1.5B): ≈ 17.5 PPL. (Section 3.6)
Mutual information between two views (CLIP)	InfoNCE contrastive loss. The InfoNCE bound on MI is capped at $\log(\text{batch size})$ (Chapter 12); CLIP-scale training uses $\geq 32\text{K}$, but queue-based methods (MoCo) and SigLIP eliminate this large-batch requirement. (Section 3.7)

CHAPTER 4

Linear Models and Their Geometry

KEY IDEA · Chapter Summary

Key results: (1) OLS: $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$; Gauss–Markov guarantees it is BLUE. (2) Ridge adds $\lambda \|\mathbf{w}\|^2$ for numerical stability and bias-variance trade-off. (3) Lasso (ℓ^1 penalty) induces sparsity; ISTA solves the general-design case in $O(1/T)$ via soft-thresholding updates. (4) SVM margin maximization leads to a convex QP dual; support vectors are the only data points that determine the boundary. (5) Rademacher complexity bounds generalization error in terms of hypothesis class richness and sample size.

Running example: OLS recovers $\hat{\mathbf{w}} \approx (5.1, 9.8)$ on the exam score data. Ridge (on centered features) shrinks both coefficients; Lasso zeroes out x_2 (GPA) first as λ grows, despite its larger true coefficient, because x_1 (study hours) has more leverage from its wider range. (On the separate 20-feature synthetic design in the exercises, ISTA with $\lambda = 0.4$ recovers the true support $\{0, 2\}$.)

DECISION GUIDE · Which Linear Model?

Task and signal structure	Model
Regression, all features likely relevant	Ridge (ℓ^2): $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. Always invertible; normal equations in $O(d^3)$. (Section 4.2)
Regression, sparse signal (feature selection)	Lasso (ℓ^1): ISTA soft-threshold $\mathcal{S}_{\eta\lambda}(u) = \text{sign}(u)(u - \eta\lambda)_+$. Exact zeros; not differentiable at 0. (Section 4.3)
Binary classification, probabilistic output	Logistic regression: $\sigma(\mathbf{w}^\top \mathbf{x})$. MLE = CE loss; loss gradient = $(\hat{p} - y)\mathbf{x}$ where $\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x})$ (negate for log-likelihood score $\mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{p}})$). (Section 4.4)
Classification, maximum margin	SVM: $\min \frac{1}{2} \ \mathbf{w}\ ^2$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$. Dual reveals that only support vectors matter. (Section 4.6)
Nonlinear boundary, small n ($< 10\text{K}$)	Kernel SVM: see Chapter 6 for the kernel trick.

CHAPTER 5

Gradient Descent and the Optimization Landscape

KEY IDEA · Chapter Summary

Key results: (1) GD on L -smooth convex functions converges at $O(1/T)$; Nesterov achieves the optimal $O(1/T^2)$ by using a momentum sequence. (2) Adam adapts learning rates per parameter using running estimates of first and second moments; bias correction makes $\hat{\mathbf{m}}^{(t)}$ and $\hat{\mathbf{v}}^{(t)}$ unbiased from step 1; without it the early steps are several times too large. (3) KKT conditions are necessary for optimality under a constraint qualification (Slater's condition for convex programs). (4) The PL condition is weaker than strong convexity but still implies linear convergence of GD.

Running example: On the unstandardized exam-score logistic regression, GD at $\eta = 0.01$ needs about 2.6×10^5 steps to close within 1% of optimal; Adam at $\eta = 0.1$ does it in ≈ 800 , because its per-coordinate scaling compensates for the intercept/ x_1/x_2 scale mismatch a single constant step size cannot.

DECISION GUIDE · Which Optimizer?

<i>Setting</i>	<i>Optimizer and key defaults</i>
Transformer (any modality)	AdamW: $\beta_1=0.9$, $\epsilon=10^{-6}$ (bf16), clip 1.0, weight decay 0.1 (exclude 1D params: LayerNorm scales/biases and all biases). $\beta_2 = 0.95$ for long training runs: lower β_2 makes the second-moment estimate more responsive to sudden gradient surges, suppressing loss spikes at the cost of slightly noisier estimates; $\beta_2 = 0.999$ gives smoother but potentially less spike-stable training. (Section 5.5; the $1/\sqrt{1-\beta_2}$ spike inflation and the $1/(1-\beta_2)$ memory window are equations (5.43) and (5.44))
CNN / ResNet	SGD + Nesterov momentum 0.9, cosine schedule, weight decay 10^{-4} . Batch size ≤ 256 ; linear LR scaling for larger batches. (Section 5.3)
Scaling from a known base shape	μ P: learning rate $\propto 1/d$, output logit scale $\propto 1/d$. Tune on a tiny proxy model; transfer to full scale. (Section 5.6)
Convex or small model ($d \lesssim 10^6$)	L-BFGS (quasi-Newton): fast linear convergence with a much better constant than GD, little tuning. Gradient storage: $2m$ vectors for memory- m variant. (Section 5.7)
Constrained problem (fairness, capacity)	KKT conditions + dual; projected gradient for simple sets. Duality gap = 0 when Slater's condition holds. (Section 5.8)

CHAPTER 6

Kernel Methods and Feature Spaces

KEY IDEA · Chapter Summary

Key results: (1) The Gram-matrix criterion (Moore–Aronszajn) characterizes valid kernels (PSD Gram matrices); Mercer’s theorem provides an eigenfunction expansion for continuous PSD kernels. (2) The representer theorem: any penalized empirical risk minimizer over an RKHS lies in the span of $\{k(\cdot, \mathbf{x}_i)\}_{i=1}^n$. (3) GP regression: the posterior mean and variance over $f(\mathbf{x}^*)$ are $\mu_*(\mathbf{x}^*) = \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$ and $\sigma_*^2 = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*$ (where σ_n^2 is the noise variance; predictive variance for a new observation y^* adds a further σ_n^2). (4) The RBF length-scale ℓ controls the effective rank of the kernel matrix and the smoothness of the fitted function.

Running example: RBF kernel ridge and RBF-kernel SVR on the exam score data match their linear counterparts (as expected for a linear DGP). The GP posterior mean recovers $\approx 5x_1 + 10x_2$; uncertainty is highest far from the training data distribution.

DECISION GUIDE · Kernels vs. Neural Networks?

Setting	Approach
$n \lesssim 10^5$, interpretability required	Kernel methods (RBF, equation (6.3)). Exact GP/KRR is $O(n^3)$, practical for $n \lesssim 10^4$ on CPU and 5×10^4 (up to $\sim 7 \times 10^4$) on one 80 GB GPU; beyond that use conjugate gradients, Nyström, or SVGP. Representer theorem: solution lives in span of training points. (Section 6.6)
Need calibrated predictive uncertainty	Gaussian Process: posterior mean = kernel ridge regression; posterior variance free. GP $\equiv$ Bayesian kernel regression. (Section 6.4)
$n > 100,000$ or raw pixels / tokens	Neural network: feature learning beats fixed kernels at scale. NTK $\rightarrow$ infinite-width NN $\equiv$ kernel ridge regression. (Section 6.7)
Feature map is known and finite-dim	Explicit features $\phi(\mathbf{x})$ when $d_\phi \lesssim n$; otherwise the kernel evaluates $\langle \phi, \phi \rangle$ in $O(d)$. (Section 6.1)
Multiple kernel sources (text + image + graph)	Multiple kernel learning: $k = \sum_i \alpha_i k_i$, $\alpha_i \geq 0$. Non-negative combination preserves PSD. (Section 6.1)

CHAPTER 7

Shallow Neural Networks

KEY IDEA · Chapter Summary

Key results: (1) The Universal Approximation Theorem (Cybenko 1989): a single hidden layer with any *continuous* sigmoidal activation is dense in $C(K)$ for any compact K . (2) Depth separation: some functions require exponentially more neurons in shallow vs. deep networks. (3) Xavier init: $\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$ keeps activation variance ≈ 1 through linear layers; He init: $2/n_{\text{in}}$ accounts for ReLU halving the forward second moment $\mathbb{E}[(\text{relu}(z))^2]$. (4) Vanishing gradients arise when $\prod_l \max |\phi'(z^{(l)})| \cdot \|\mathbf{W}^{(l)}\|_2 \ll 1$; LSTM gates and residual connections are architectural fixes.

Running example: A 2-layer ReLU network with He initialization trained on the exam score data achieves training loss within 5% of the OLS solution. In the separate 20-layer signal-propagation experiment (see the computational exercise), Xavier initialization loses $\approx 99.9\%$ of the activations' second moment by layer 10, the same halving pattern computed above.

DECISION GUIDE · *Activation Functions and Shallow Architecture*

<i>Setting</i>	<i>Choice and rationale</i>
General hidden layer, default choice	ReLU: $\text{ReLU}(x) = \max(0, x)$. Sparse, fast, no vanishing gradient for $x > 0$. Risk: dying neurons with excessively large learning rates or poor weight initialization. (Section 7.2)
Dead ReLU observed ($> 5\%$ of neurons inactive across all training inputs)	Leaky ReLU ($\alpha=0.01$) or GELU. $\text{GELU} = x \cdot \Phi(x)$; smoother, preferred in NLP. (Section 7.2)
Transformer FFN (language or vision)	SwiGLU or GELU. SwiGLU uses <i>three</i> weight matrices ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$): output $= W_{\text{down}}(\text{SiLU}(W_{\text{gate}}\mathbf{x}) \odot W_{\text{up}}\mathbf{x})$; net params $\approx 8d^2$ per block when hidden dim $= \frac{8d}{3}$. (defined in Section 11.6)
Width vs. depth trade-off	UAT gives no explicit width bound (only existence). Barron's theorem: $O(\varepsilon^{-2})$ neurons suffice for L^2 approximation of functions with bounded Fourier magnitude (though $C_f^2 = (\int \ \boldsymbol{\omega}\ \hat{f}(\boldsymbol{\omega}) d\boldsymbol{\omega})^2$ may grow with input dimension for specific function classes). Depth reduces width exponentially for compositional functions. (Section 7.1)
Output layer	Sigmoid (binary), softmax (multi-class), linear (regression). Avoid plain sigmoid as a hidden activation in deep stacks (vanishing gradients); sigmoid and softmax inside hidden layers remain standard as gates and attention weights.

CHAPTER 8

Deep Networks: Backpropagation and Modern Architectures

KEY IDEA · Chapter Summary

Key results: (1) Reverse-mode automatic differentiation computes all gradients for a small constant multiple of the forward-pass cost — and, since the forward pass itself is linear in parameter count, the weight-gradient computation is linear in the number of parameters. (2) Batch normalization stabilizes training by standardizing pre-activation distributions; its backward pass requires careful tracking of the mean and variance as intermediate variables. (3) Residual connections create a gradient highway: the identity term in each factor ($\mathbf{I} + \mathbf{J}_k$) gives the gradient a direct path to every layer, and in the $S < \ln 2$ regime rules out *exponential* decay. (4) Dropout regularizes by preventing co-adaptation; it approximates Bayesian inference over weight distributions. **Running example:** stacking BatchNorm and a residual skip onto the two-layer exam-score predictor gives every residual factor the form $(\mathbf{I} + \mathbf{J}_k)$, so the gradient keeps an identity path to every layer. In the $S < \ln 2$ regime of Section 8.6 this rules out exponential decay.

DECISION GUIDE · Normalization, Depth, and Precision

<i>Setting</i>	<i>Choice and rationale</i>
CNN, batch size ≥ 32	BatchNorm: normalizes across batch dimension. Biased at small batches (< 8); switch to GroupNorm when batches are small (detection/segmentation); do not use with variable-length sequences. (Section 8.4)
Transformer or variable-length sequences	LayerNorm / RMSNorm: normalizes across feature dimension. Pre-LN (before sublayer) is standard for stability. (Section 8.5)
Initialization for ReLU networks	He/Kaiming: $\sigma^2 = 2/n_{\text{in}}$. Xavier for tanh: $\sigma^2 = 2/(n_{\text{in}} + n_{\text{out}})$; the Glorot/Theano heuristic rescales Xavier by $1/\phi'(0)^2 = 16$ for sigmoid (Section 7.6).
Residual network training precision	bf16: 8-bit exponent, no overflow at gradient scale. fp16: needs dynamic loss scaler; exp overflows when input $\gtrsim \ln(65504) \approx 11$. (Section 8.9)
Gradient vanishing in deep (> 20 layer) network	Add residual connections $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Gradient highway: $\partial\mathcal{L}/\partial\mathbf{x} = \partial\mathcal{L}/\partial\mathbf{y}(\mathbf{I} + \partial\mathcal{F}/\partial\mathbf{x})$ (Section 8.6)

CHAPTER 9

Convolutional Neural Networks

KEY IDEA · Chapter Summary

Key results: (1) Convolution implements translation equivariance: $\text{Conv}(T_\delta x) = T_\delta \text{Conv}(x)$. (2) As a matrix-vector product, 1D convolution is Toeplitz; 2D (finite/-valid) is doubly block-Toeplitz, and the doubly block-circulant (circular boundary) variant diagonalizes under the 2D DFT, enabling FFT-based computation in $O(HW \log HW)$ after zero-padding. (3) Gradient w.r.t. input: full cross-correlation with flipped kernel; w.r.t. kernel: valid cross-correlation of input with output gradient. (4) Receptive field grows as $r_l = r_{l-1} + (k_l - 1) \prod_{i=1}^{l-1} s_i$; dilation multiplies the effective kernel span without adding parameters.

Running example: A 1D convolution of the 10-day study log with kernel $[0.5, 1.0, 1.5]$ (weighting recent days more) produces a “weighted momentum” feature; whether it helps depends on how the study log enters the (so far linear, two-feature) score DGP, which this chapter does not specify.

DECISION GUIDE · CNN Architecture Choices

<i>Goal</i>	<i>Design choice</i>
Image classification, compute budget normal	Standard conv stack: 3×3 kernels, stride-2 downsampling, global average pooling at output. Params per conv layer = $k^2 C_{\text{in}} C_{\text{out}}$. (Section 9.1)
Efficient inference (mobile, edge)	Depthwise-separable: $k^2 C_{\text{in}}$ (depthwise) + $C_{\text{in}} C_{\text{out}}$ (pointwise). $\approx 8-9\times$ fewer ops than standard 3×3 conv. (Section 9.5)
Multi-scale feature extraction (detection)	Feature pyramid network: share backbone, lightweight heads at each stride level ($\times 4, \times 8, \times 16, \times 32$). A top-down pathway upsamples coarse, semantically strong features and adds them to same-resolution backbone features through lateral 1×1 connections, so every level detects with strong semantics, not just the deepest one.
Translation invariance wanted	Global average pooling removes spatial dimensions; more invariant than fully connected.
Gradient vanishing in very deep CNN	Residual connections (ResNet, additive skips) or dense connections (DenseNet, where each layer concatenates <i>all</i> earlier feature maps as input, so every layer sits one edge from the loss and reuses features instead of relearning them, at the cost of a growing concatenation buffer).

CHAPTER 10

Recurrent Networks: Sequence Modeling Through Time

KEY IDEA · Chapter Summary

Key results: (1) BPTT gradient:

$$\partial \mathcal{L} / \partial \mathbf{h}_s = \sum_{t=s}^T \left[\prod_{i=s+1}^t \mathbf{W}_{hh}^\top \text{diag}(\phi'(z_i)) \right] \boldsymbol{\delta}_t^{\text{out}},$$

where $\boldsymbol{\delta}_t^{\text{out}} = \partial \mathcal{L}_t / \partial \mathbf{h}_t$ is the per-step direct loss gradient at step t (equation (10.5); standard left-to-right product; the $t=s$ term has an empty product equal to $\mathbf{I}$); vanishes (sufficient: $\sigma_{\max}(\mathbf{W}_{hh}) \cdot |\phi'|_{\max} < 1$; for $\tanh$ where $|\phi'| \leq 1$, $\sigma_{\max} < 1$ alone suffices), explodes in the worst case over admissible activation sequences ($c \cdot \rho(\mathbf{W}_{hh}) > 1$ where $c = \inf_j \phi'_j > 0$, i.e. the activation is non-saturating). (2) LSTM cell state: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$; gradient highway (CEC approximation, gates frozen): $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx \text{diag}(\mathbf{f}_t)$ avoids the product-of- $\mathbf{W}_{hh}$ problem. The exact Jacobian (equation (10.24)) adds three $\mathbf{W}_{*,h}$ -carrying terms scaled by $\|\mathbf{o}_{t-1} \odot (\mathbf{1} - \tanh^2(\mathbf{c}_{t-1}))\|_\infty$, so the highway is an operating regime (saturated cell, open forget gate) rather than an identity (equation (10.29)). (3) Gradient clipping = projection onto $\{\|\mathbf{g}\| \leq \tau\}$; bounds step size to $\eta\tau$. (4) GRU update is an Euler step with adaptive step size z_t .

Running example: The LSTM processes the 10-day study log left-to-right; the cell state highway can carry the running total of effective study hours across all 10 days if the forget gate stays near 1; what it learns depends on how the study log enters the score DGP, which this chapter does not specify.

DECISION GUIDE · *RNN vs. LSTM vs. Transformer*

<i>Setting</i>	<i>Architecture</i>
Memory-constrained or long-sequence streaming	LSTM or GRU: constant memory, $O(1)$ per-step compute (no growing KV cache). Lower per-step cost than Transformers when sequence length is long; small Transformers with KV cache are often competitive on short sequences. GRU has $\approx 25\%$ fewer params than LSTM; quality is task-dependent. (Section 10.5)
Sequence length > 512 , global context required	Transformer: $O(n^2d)$ time complexity; FlashAttention reduces extra memory from $O(n^2)$ to $O(n)$ (and cuts HBM traffic) via tiled computation (time complexity remains $O(n^2)$); sparse attention reduces time complexity. Scales poorly beyond 32K tokens without one of these. (Chapter 11)
Gradient vanishing observed (loss stalls early)	Switch to LSTM or GRU (gating prevents vanishing); gradient clipping (clip 1.0) and BPTT truncation address exploding, not vanishing, gradients. (Section 10.3)
Online / causal processing, tiny model ($< 1\text{M}$ params)	GRU with hidden $d = 256$; runs on CPU in real time. LSTM if gate fidelity matters more than speed. (Section 10.5)
Time series with known periodicity	Vanilla RNN may suffice if the period is short relative to the gradient horizon; add skip connections across periods (lag- P inputs) so each period is one recurrent step away.

CHAPTER 11

Attention and Transformers

KEY IDEA · Chapter Summary

Key results: (1) Scaled dot-product attention: $\text{Attn}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k})V$; scaling by $\sqrt{d_k}$ prevents softmax saturation in high dimensions. (2) Multi-head attention applies H independent attention heads in parallel; total parameter count per layer is $4d^2$ regardless of H . (3) FlashAttention tiles the attention computation in SRAM to achieve $O(nd)$ activation memory (eliminating the $O(n^2)$ attention matrix) and $\Theta(n^2d^2/M)$ HBM I/O accesses (vs. $\Theta(n^2 + nd)$ for standard attention); output equals standard attention up to floating-point rounding order. (4) RoPE encodes relative positions via frequency-dependent rotations; $\langle \text{RoPE}(q, m), \text{RoPE}(k, n) \rangle$ depends only on $n - m$.

Running example: Self-attention over the 10-day study log; the attention weight A_{ij} quantifies how much day- i output depends on day- j study hours. In the toy example (example 11.1) the weights favor recent days; the LSTM (Chapter 10) can instead carry the running total along its cell path if the forget gate stays near 1.

DECISION GUIDE · Which Attention Variant?

<i>Setting</i>	<i>Choice and defaults</i>
Training a new Transformer from scratch	GQA ($G = 8$) + FlashAttention (plain MHA only for small models where the KV cache is irrelevant). Verify with the <code>sdpa_kernel(SDPBackend.FLASH_ATTENTION)</code> context manager (errors if flash cannot run). Always set $d_k = d/H$; use $1/\sqrt{d_k}$ scaling. (Sections 11.2 and 11.9)
Inference throughput is the bottleneck	GQA with $G = 8$ KV groups: $H/G \times$ more concurrent requests. KV cache per request: $2 \cdot L \cdot G \cdot d_k \cdot n \cdot 2$ bytes. (Section 11.10.2)
Context window > 8K tokens	RoPE base matched to training (500,000 for Llama-3) plus the model's <code>rope_scaling</code> ; never the framework default. (Section 11.5)
Memory-constrained serving (single GPU)	GQA + INT4 quantization + paged KV cache (<code>gpu_memory_utilization=0.90</code>). Llama-3-70B INT4: ≈ 35 GB weights. (Section 11.10.1)
Document / sliding-window task	Sliding-window attention: Mistral $w = 4,096$. Fixed KV memory regardless of total sequence length. (Section 11.11)

CHAPTER 12

Generative Models

KEY IDEA · Chapter Summary

Key results: (1) ELBO: $\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$; the KL term has a closed form for Gaussian q and prior. (2) DDPM: the forward marginal is $q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$; the reverse process learns to denoise by predicting the noise $\boldsymbol{\varepsilon}$. (3) CFG: $\tilde{\boldsymbol{\varepsilon}} = (1 - w)\boldsymbol{\varepsilon}_\emptyset + w\boldsymbol{\varepsilon}_c$; larger w increases condition adherence at the cost of diversity. (4) GAN training at equilibrium minimizes JS divergence; Wasserstein GAN replaces JS with the smoother W_1 distance.

Running example: Fit concretely, $K = 2$ EM on the exam score dataset splits students by study hours alone (the dataset’s high-variance feature, per Chapter 1’s SVD), not by a behavioral archetype; a VAE on the same two-feature data has no compression to do with a 2D latent, since ambient and latent dimension coincide.

DECISION GUIDE · Which Generative Model?

Goal	Model and key defaults
Image synthesis, quality primary concern	Diffusion (DDPM/LDM): $T = 1,000$ diffusion timesteps; DDIM inference in 50 steps. $\approx 20\times$ faster than DDPM sampling. (Section 12.6)
Fast inference required (~ 8 – 20 steps)	Flow matching: straight-line ODE paths, 8–20 NFE. (Section 12.6.10)
Structured latent space or VAE bottleneck needed	VAE: ELBO = reconstruction $- \beta \cdot \text{KL}$. Monitor per-dim KL; posterior collapse if more than half the dimensions sit at $\text{KL}_j \approx 0$. (Section 12.2)
Classifier-free guidance (conditional generation)	CFG: $\tilde{\boldsymbol{\varepsilon}} = (1 - w)\boldsymbol{\varepsilon}_\emptyset + w\boldsymbol{\varepsilon}_c$ ($w = 7.5$ is Stable Diffusion’s guidance_scale ; this equals $\boldsymbol{\varepsilon}_\emptyset + w(\boldsymbol{\varepsilon}_c - \boldsymbol{\varepsilon}_\emptyset)$). Condition dropout 10–20% during training. (Section 12.6)
Zero-shot image–text alignment	CLIP contrastive loss: learned temperature, initially $\tau = 0.07$; batch $\geq 32\text{K}$. InfoNCE quality $\propto \log(\text{batch size})$. (Section 12.7)

CHAPTER 13

Reinforcement Learning: Optimization Under Uncertainty

KEY IDEA · Chapter Summary

Key results: (1) Bellman optimality: $Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s', a')]$; uniqueness and convergence follow from the γ -contraction property. (2) Policy gradient theorem: $\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t]$ (with $\gamma = 1$ for the episodic case, recovering $\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$); adding a baseline $b(s_t)$ reduces variance without biasing the gradient. (3) DPO: closed-form solution to the KL-constrained RLHF problem, expressed as a log-ratio loss between preferred and rejected responses.

Running example: Study scheduling as an MDP: state (t , cumulative hours), action hours on day t , reward exam score (a function of the 10-day total only). The optimal policy maximizes total effective hours under that reward; because the reward does not see the day-by-day schedule, this particular MDP cannot by itself explain *why* spaced practice beats cramming — that would need a reward sensitive to timing, not just the sum.

DECISION GUIDE · Which Alignment Algorithm?

Setting	Algorithm and key defaults
RLHF with a trained reward model	PPO: clip $\epsilon = 0.2$; KL penalty $\beta = 0.02$ – 0.2 . Expensive (3 weight sets: shared policy+value backbone, frozen reference, reward model). Use when reward model is well-calibrated. (Section 13.6)
Preference pairs available, no reward model	DPO: $\beta = 0.1$ (KL weight); reference model frozen. Simpler, fewer GPUs; verify chosen > rejected before training. (Section 13.8)
Reasoning / math / code (verifiable rewards)	GRPO: group size $G = 16$ rollouts per prompt (math/reasoning tasks). No value network needed; normalize rewards within group. (Section 13.9)
Supervised fine-tune only (no alignment)	SFT: CE loss on demonstrations. Mask prompt tokens; causal mask prevents look-ahead. (Section 11.7)
Over-optimization of reward proxy suspected	Monitor held-out quality while reward increases. Reward saturation + quality drop = Goodhart’s law triggered. (Section 13.7)

CHAPTER 14

Frontiers: Where Mathematics Meets Research

KEY IDEA · Chapter Summary

Key results: (1) Chinchilla scaling: in the symmetric approximation, for fixed compute C , optimal training uses $N^* \propto C^{1/2}$ parameters and $D^* \propto C^{1/2}$ tokens, giving the familiar ratio $D^*/N^* \approx 20$ tokens per parameter — a practitioner’s shorthand close to Hoffmann et al.’s Approaches 1–2 ($\approx 0.50/0.50$); their parametric fit (Approach 3) is mildly asymmetric ($\approx 0.46/0.54$) and, with its published constants, asks for a smaller model at Chinchilla’s budget (with the unrounded fit $\alpha = 0.339$, $\beta = 0.285$: ≈ 40.7 B parameters and 59 tokens per parameter). (2) Neural ODE: the continuous limit of residual networks; adjoint method computes gradients in $O(1)$ memory via a backward ODE. (3) LoRA: rank- r fine-tuning adds $2dr$ parameters (versus d^2 for full fine-tuning); initialize $\Delta \mathbf{W} = \mathbf{B}\mathbf{A} = \mathbf{0}$. (4) SSMs: S4 uses parallel FFT convolution during training (enabled by its linear time-invariant structure); Mamba uses a parallel associative scan during training and a sequential RNN during inference (input-dependent parameters break LTI, so FFT convolution does not apply); associativity of the state-transition operator enables $O(\log N)$ parallel computation.

Running example: Natural gradient descent on the exam-score logistic regression preconditions by the Fisher information matrix, undoing the sigmoid’s saturation warp and the unequal feature scales that slow standard gradient descent (equation (14.12)).

What we have not covered: Causal inference and structural equation models; differential privacy and DP-SGD; conformal prediction. (Sparse autoencoders and circuit analysis are the subject of Chapter 16; calibration monitoring is in Chapter 19.) See the Further Reading for pointers. (Contrastive objectives, including CLIP, are covered in §12.7; test-time compute appears as process reward models and Best-of- N in Chapter 13 and as speculative decoding in §14.11.)

DECISION GUIDE · *Scaling and Architecture at the Frontier*

<i>Constraint</i>	<i>Strategy</i>
Fixed compute budget, choose data vs. model size	Chinchilla: $D \approx 20N$ tokens for compute-optimal run. Both N and D scale as $C^{0.5}$. (Section 14.6)
Data-constrained (cannot get $20N$ tokens)	Repeat deduplicated data or train a smaller N . Up to ≈ 4 epochs of repeated data are nearly as good as fresh data, with rapidly diminishing value beyond (Muennighoff et al. 2023).
Memory-constrained, need $> 7B$ model on 1 GPU	INT4 (GPTQ/AWQ) (Frantar et al. 2023; Lin et al. 2024): $4\times$ memory reduction; reported PPL loss is <1 for $\geq 30B$ models and usually still under 1 at 7B with GPTQ/AWQ (naive rounding degrades more).
Long-range dependencies $> 32K$ tokens	State-space model (Mamba): $O(N)$ vs. $O(N^2)$. SSM selectivity: input-dependent $\mathbf{B}, \mathbf{C}$ matrices and step size Δ ; $\mathbf{A}$ is a fixed learned parameter. (Section 14.9)
Mixture of experts, sparse routing	MoE: top- k routing ($k=1-8$), load balancing (auxiliary loss or bias-based). Active params $\approx k/E$ of total; communication cost across GPUs. (Section 14.8)

CHAPTER 15

Production Inference Economics

KEY IDEA · Chapter Summary

Key results:

1. Prefill is compute-bound; decode is memory-bound. The same hardware runs at vastly different efficiencies in the two regimes; serving design must respect this.
2. Continuous batching with paged attention is the dominant scheduling and memory pattern. Both are non-negotiable for cost-competitive serving.
3. As of early 2026, INT4 weight quantization (GPTQ, AWQ) is a common default for production LLM serving; FP8 KV cache halves the KV memory footprint on supported hardware. Recompute which format wins for your stack using §14.10 — the dominant format shifts as hardware adds native low-precision support.
4. Speculative decoding helps only when decode is memory-bound; its marginal value shrinks as batch occupancy rises.
5. The cost-per-token equation $\$/\text{M tok} = C_{\text{GPU}} / (3600 \cdot T_{\text{out}} \cdot U) \cdot 10^6$ is the single equation that closes capacity-planning and unit-economics conversations.
6. Compound systems (routing, prompt caches, result caches) add a multiplicative cost reduction on top of single-model optimizations.

Running example: None. Inference economics is a systems question about real serving stacks, which the exam-score dataset cannot illustrate.

Where this leaves you: This chapter closes the economics arc. With everything from the roofline model to equation (15.12), you can now price an end-to-end product build (a tokenizer, a 1B-parameter pretrain, an RLHF alignment loop, and the serving stack that puts everything together), and building one is a question of engineering time rather than missing theory. The nanoGPT capstone at the end of Chapter 11 scales the pretraining piece down to something you can run. It is a character-level model with no tokenizer, RLHF, or serving stack, so treat it as a hands-on anchor for the pretraining cost arithmetic above rather than a build of the whole pipeline. What the cost equation cannot tell you is whether the model you are serving so cheaply actually learned what you think it learned; that is the question Chapter 16 takes up next, closing Part IV. Part V then turns from mathematics to engineering practice — abstraction, testing, versioning, and observability: the disciplines that keep a profitable serving stack alive in production.

DECISION GUIDE · Which Inference Optimization?

<i>Bottleneck or constraint</i>	<i>Optimization</i>
Memory-bandwidth bound (most LLMs in decode)	Quantize weights to INT4/INT8: reduce bytes read per token. Roofline: op intensity < 156 FLOP/byte (A100-80GB SXM) / 295 FLOP/byte (H100) $\Rightarrow$ bandwidth-bound. (Section 15.1)
Low throughput from static or no batching	Continuous batching (vLLM/TGI): $\approx 17\times$ throughput vs. batch-1 serving (equation (15.7)). Set <code>gpu_memory_utilization = 0.85–0.90</code> . (Section 15.5)
Decode (TPOT) tail spikes when long prefills arrive	Prefill/decode disaggregation: dedicate GPUs to each phase. Prefill is compute-bound; decode is memory-bound. (Section 15.4)
Need $2\times$ decode speedup with same model quality	Speculative decoding: use small draft model; accept if draft model's acceptance rate ≥ 0.7 at typical cost ratios — at $c = 0.1$, $\bar{\alpha} = 0.6$ tops out near $1.7\times$ (a 1B draft with $c \approx 0.014$ still reaches $2.2\times$). (Section 14.11)
Serving multiple LoRA adapters	Keep adapters separate (hot-swap) only if tasks change per request. Otherwise merge $\Delta\mathbf{W} = \mathbf{B}\mathbf{A}$ into $\mathbf{W}_0$ before export. (Section 11.11.2)

CHAPTER 16

Mechanistic Interpretability

KEY IDEA · Chapter Summary**Key results:**

1. **Superposition:** the near-orthogonal packing theorem (Theorem 16.1, a cousin of the classical Johnson–Lindenstrauss projection lemma) proves a d -dimensional layer can represent exponentially more than d nearly orthogonal feature directions, explaining why polysemanticity is geometrically cheap rather than a training failure, though the random-direction bound likely underestimates a trained model’s actual packing.
2. **Sparse autoencoders** solve equation (16.5), a nonnegativity-constrained relative of the Lasso objective of Chapter 4, whose coordinate-wise solution is the *one-sided* soft-threshold $\mathcal{S}_\lambda^+$ of equation (16.6), not the two-sided $\mathcal{S}_\lambda$; dead latents and feature absorption are known, partially mitigated failure modes.
3. **Transcoders and cross-layer transcoders** reconstruct an MLP sublayer’s output from a sparse code of its input rather than reconstructing the input itself (a CLT lets each code write into every later layer), making cross-layer circuit structure visible in a way single-layer SAEs structurally cannot.
4. **A circuit**, operationally, is a minimal set of components whose patched activations jointly explain a behavior on tested inputs; induction heads (a two-head composition implementing $[A][B] \dots [A] \rightarrow [B]$) are the chapter’s worked example, with causal evidence in small models and correlational evidence in larger ones.
5. **A difference-of-means steering vector** is the direction maximizing the mean-activation gap between two contrastive prompt sets, a direct corollary of Chapter 1’s inner-product geometry.
6. **Chain-of-thought faithfulness** varies widely by task; on this chapter’s reading it is strongest on hard tasks the model cannot skip steps on and weakest on easy tasks. It is structurally time-limited if latent reasoning displaces token-level traces.

Running example: None. Interpretability requires internal structure a two-feature logistic regression does not have; this chapter follows Chapter 15’s precedent of breaking the throughline for the same reason, using small attention-only toy transformers instead.

What we have not covered: causal-abstraction theory beyond the operational definition of activation patching given here; interpretability for non-language modalities (vision, audio, scientific foundation models), an active and separate research thread. See the Further Reading for pointers.

DECISION GUIDE · *Choosing an Interpretability Tool*

Question	Tool
Does this layer represent concept X at all?	Single-layer SAE: fit equation (16.5), check which latent fires. Cheap, fast, structurally blind to cross-layer composition. (§16.2.1)
How does the model connect X (early layer) to Y (late layer)?	Transcoder or CLT: the dictionary method that can represent a feature computed across multiple layers. (§16.2.2)
I already suspect factor X caused this specific output — confirm it.	Activation patching: patch the one candidate activation, measure restored output difference. Requires a hypothesis in hand. (§16.3.2)
I have a wrong output and no hypothesis yet.	Attribution graph: generates candidate factors from the model’s own computation; confirm the strongest candidate with a targeted patch. (§16.3.3)
I want to nudge a behavior at inference time, no retraining.	Difference-of-means steering vector, gated to fire only on matching inputs. (§16.3.4)
Can I trust this model’s stated chain-of-thought as a monitoring signal?	Only on tasks hard enough that the model cannot skip the steps; check faithfulness and verbosity separately, and note this channel disappears under latent reasoning. (§16.4)

CHAPTER 17

Abstraction and Interface Design in ML Systems

KEY IDEA · Chapter Summary

Key results:

1. A production ML system is a stack of five layers (data, features, training, serving, monitoring), and each layer fails with a recognizably different signature (Section 17.1).
2. CACE: in an ML system a stable interface is necessary but not sufficient. The statistical properties of the values crossing an interface are part of the model's real contract, even though no compiler enforces them (Section 17.2).
3. Training/serving skew has four recurring causes (duplicate implementations, point-in-time violations, default divergence, staleness); point-in-time joins keep training features from seeing values that were not yet available (Sections 17.2.3 and 17.2.4).
4. Code, configuration, and data are separate concerns, and a notebook that tangles them is safe to explore in but not to deploy (Section 17.3).
5. A contract between stages should encode type, nullability, valid range, keys, units, and freshness bounds. Distributional comparisons should warn rather than gate, contract tests pin down meaning with fixed examples, and a field whose meaning changes gets a new version instead of an in-place redefinition (Section 17.4).

DECISION GUIDE · Which Skew Cause Does This Symptom Point To?

Observed symptom	Likely cause
A single feature's serving-time value never quite matches its offline-computed value on the same underlying record, even when both were computed "at the same time," and the gap is a rounding difference, a null-handling difference, or a categorical encoded differently.	Two implementations of the same logic (batch pipeline vs. low-latency path) have diverged.
Offline validation accuracy looks implausibly strong, and the drop only shows up once the model is serving live traffic, not caught by any train/test split, however it was drawn.	Point-in-time correctness violation: a training-time feature saw information from after its label's timestamp, so offline evaluation is leaking the same way training did.
A specific missing-value code path behaves differently offline vs. online, but only for records that hit that path (a rare category, a cold-start user), and is otherwise invisible in aggregate metrics.	Silent default divergence: training and serving impute the same logical gap with two different concrete values.
Model quality degrades gradually over the days after each retrain and then recovers right after the next one, or degrades specifically for requests served long after their inputs were computed.	Staleness / time-basis skew: training saw one refresh cadence or time reference for a feature and serving supplies another.

DECISION GUIDE · Which Contract Enforcement Mechanism?

Situation	Reach for
A single team owns both sides of the boundary; the risk is a typo or an accidental type change.	Lightweight schema validation at the boundary (types, nullability, ranges), run on every pipeline execution.
Two teams own opposite sides; changes are deployed independently and on different schedules.	Explicit, versioned schema <i>plus</i> a small set of contract tests with concrete example inputs/outputs, run in the producer's CI so a breaking change is caught before it ships, not after.
The risk is not shape but <i>meaning</i> drift in a field whose type never changes (a redefined business metric, a re-scoped boolean flag).	Contract tests with fixed, known example cases; schema validation alone will not catch this.
The feature crosses the offline/online (training/serving) boundary specifically.	A single shared feature computation (feature-store pattern, Section 17.2.3) so there is one definition to enforce a contract on, not two to keep in sync.
The boundary is high-stakes (financial, safety-, or fairness-relevant) and low-change-frequency.	All of the above, plus mandatory human review of any change that touches the contract, with the contract's current definition kept in a location visible to both producer and consumer teams.

CHAPTER 18

Testing, Versioning, and Reproducibility for ML Engineers

KEY IDEA · Chapter Summary

Key results:

1. The ML testing pyramid has four layers: unit tests, data validation tests, model behavioral tests, and integration tests. The middle two are where ML-specific failures live and are the layers most teams leave empty (Section 18.1).
2. The ML Test Score scores 28 tests in four categories on a 0 / 0.5 / 1 scale, and the overall score is the *minimum* across categories, so the weakest category sets the grade (Section 18.2).
3. Reproducibility needs three versioning axes pinned together: code, data, and model. Data should be versioned as immutable, content-hashed snapshots, not as a query against a live upstream source (Section 18.3).
4. The right amount of versioning tooling depends on team size and data scale (Section 18.4).
5. Reproducibility breaks through nondeterministic kernels, dependency and environment drift, unpinned data ordering, seed and configuration mismanagement, distributed race conditions, and notebook-to-pipeline drift (Section 18.5).

DECISION GUIDE · Which Versioning Approach?

Situation	Reasonable starting approach
Small team (1–3 ML engineers), data fits comfortably on one machine or in one cloud bucket, few retrains per month.	Content-hashed data snapshots (a manifest file plus immutable storage locations) and a lightweight, self-hosted experiment tracker that the training script writes to automatically. Prefer it to a hand-updated spreadsheet: a log someone must remember to update decays silently, and the tracker costs about the same to set up.
Growing team (roughly 4–15 ML engineers), data scale in the tens to low hundreds of gigabytes, multiple concurrent experiments per week.	A dedicated data version control tool integrated with the existing code version control system, plus an experiment tracker logging configuration and metrics per run, plus a real model registry with lineage fields (code commit, data hash, config) enforced at registration time.
Large team (15+ ML engineers) or data scale in the terabytes-plus range, or multiple teams sharing upstream datasets and features.	A feature store or equivalent shared, versioned data layer so multiple teams consume the same governed snapshots rather than each re-deriving their own; a model registry with enforced approval gates before promotion to serving; automated lineage capture (not manually filled-in meta-data) so traceability survives turnover on the team.
Regulated domain (finance, healthcare, safety-critical) at any team size.	Treat the traceability requirements of the “large team” row as a floor regardless of actual team size. Audit and compliance obligations do not scale down with headcount, and the cost of retrofitting lineage capture after a regulator asks for it is far higher than building it in from the first production model.

CHAPTER 19

Technical Debt and Observability in Production ML

KEY IDEA · *What Part V Adds to Parts I–IV*

Parts I–IV answer, with mathematical precision, the question *is this model correct*: correct in its derivation, correct in its optimization, correct in what its architecture can and cannot represent. That question has a well-posed, checkable answer, and getting it right is necessary — no amount of operational discipline rescues a model built on a broken premise. Part V answers a different question, one with no equally clean mathematical closed form: *will this correct model stay correct, stay legible, and stay owned, as it runs inside a real organization, on real data, under real organizational and economic pressure, for longer than the person who built it will personally be watching it*. Only the first one has occupied this book until now, and a reader who leaves with a precise answer to the first and no vocabulary at all for the second has learned to build systems that are correct on the day they ship and silently wrong at some unknown point after that.

DECISION GUIDE · *Which Debt Pattern Am I Looking At?*

Symptom	Likely pattern and first move
A small, seemingly isolated change (drop a feature, adjust a constant) moves metrics on slices that have nothing obviously to do with the change.	Entanglement (CACE). Check slice-level, not just aggregate, effects before and after; treat any change as potentially global until proven local.
Offline metrics are flat or improving, but an independent measure of output diversity or coverage is narrowing over many retrain cycles.	Hidden or direct feedback loop. Check whether the evaluation data is itself drawn from the model’s own past decisions; introduce held-out, exploration, or model-independent evaluation traffic.
A change that should be simple touches many repositories and scheduled jobs owned by different teams, and no one fully understands the current data flow.	Pipeline jungle. Map the actual current data flow (not the original design doc); assign explicit ownership of the end-to-end shape, not just individual scripts.
A purely internal-looking change to an output’s format, scale, or cadence breaks a system the owning team did not know was reading it.	Undeclared consumer. Audit read access to every output surface; convert the output into a declared, versioned interface with a registration step for new consumers.

DECISION GUIDE · Which Signal Catches Which Failure?

If this changed upstream...**...this is the axis that catches it**

The kind of input the model receives (a schema, a source, a user population) shifted, but nothing about how the model itself relates confidence to correctness.

Input distribution drift (Section 19.4.1).

The model's own outputs, as a distribution, look different from before — even if you cannot yet say whether inputs changed or the model itself did.

Prediction distribution shift (Section 19.4.2).

A feature is computed correctly and passes every schema check, but is older, relative to the moment of use, than the freshness the model was trained and validated under.

Feature staleness (Section 19.4.3).

The model is as accurate as ever on average, but a stated confidence of 90% stops corresponding to being right 90% of the time.

Calibration decay (Section 19.5.2).

The full derivations

Every result on these pages is derived in full in *Math for ML Engineers: From Backprop to RLHF*: 19 chapters, six appendices with 36 worked interview derivations, and tested NumPy code for every major algorithm in the free companion repo.

Book, editions and the free Chapter 5: mikaelyemane.github.io/ml-math-code

Code: github.com/mikaelyemane/ml-math-code